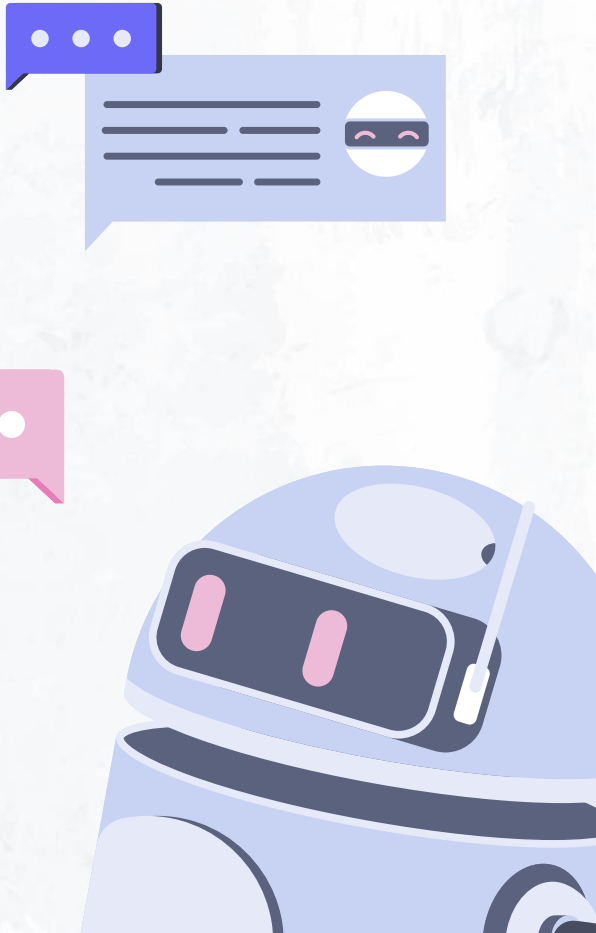
Natural Language Processing

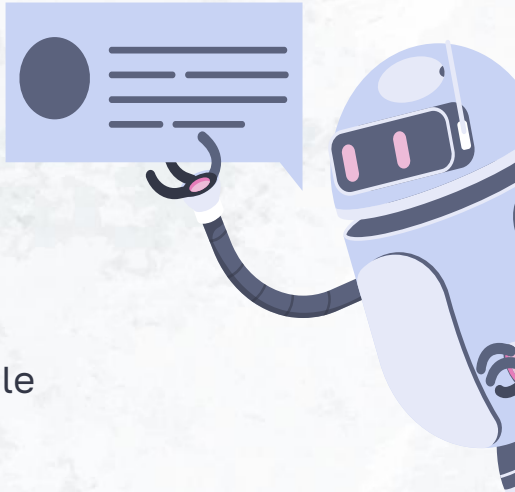


Section 2

01 →

One Hot Encoding

What is One Hot Encoding?



is a simple technique used to represent words or labels as numerical vectors.

One-hot encoding represents each word as a binary vector with a single **1** and the rest **0**, where the vector length equals the vocabulary size.

- Easy to understand and implement
- Works well for small vocabularies but becomes inefficient for large ones

Build ideas with logic and code

One Hot Encoding

Build	1	0	0	0	0	0
ideas	0	1	0	0	0	0
with	0	0	1	0	0	0
logic	0	0	0	1	0	0
and	0	0	0	0	1	0
code	0	0	0	0	0	1

Code

Step 1: Import Required Libraries.

```
import numpy as np
import string
import matplotlib.pyplot as plt
```

Step 2: Define the Corpus

```
corpus = [
    "The quick brown fox jumps over the lazy dog",
    "The dog chased the fox",
    "The fox is quick and smart"
]
```

Code

Step 3: Preprocess Text

```
def preprocess_text(text):
    text = text.lower()
    text = text.translate(str.maketrans("", "", string.punctuation))
    return text.split()
tokenized_corpus = [preprocess_text(sentence) for sentence in corpus]
```

Step 4: Build Vocabulary

```
vocabulary = sorted(set(word for sentence in tokenized_corpus for word in
sentence))
word_to_index = {word: idx for idx, word in enumerate(vocabulary)}
```

Step 5: One-Hot Encode Sentences

```python
def one_hot_encode(sentence, word_to_index):
    vocab_size = len(word_to_index)
    encoded_sentence = []
    for word in sentence:
        vector = np.zeros(vocab_size, dtype=int)
        vector[word_to_index[word]] = 1
        encoded_sentence.append(vector)
    return np.array(encoded_sentence)
encoded_vectors = one_hot_encode(tokenized_corpus[0], word_to_index)
```

Step 6: Display Results

```python
print("Vocabulary:")
print(vocabulary)

print("\nOne-Hot Encoded Vectors (First Sentence):")
for word, vector in zip(tokenized_corpus[0], encoded_vectors):
    print(f"{word}: {vector}")
```

Output

```
Vocabulary:
['and', 'brown', 'chased', 'dog', 'fox', 'is', 'jumps', 'lazy', 'over', 'quick', 'smart', 'the']

One-Hot Encoded Vectors (First Sentence):
the: [0 0 0 0 0 0 0 0 0 0 0 1]
quick: [0 0 0 0 0 0 0 0 0 1 0 0]
brown: [0 1 0 0 0 0 0 0 0 0 0 0]
fox: [0 0 0 0 1 0 0 0 0 0 0 0]
jumps: [0 0 0 0 0 0 1 0 0 0 0 0]
over: [0 0 0 0 0 0 0 0 1 0 0 0]
the: [0 0 0 0 0 0 0 0 0 0 0 1]
lazy: [0 0 0 0 0 0 0 1 0 0 0 0]
dog: [0 0 0 1 0 0 0 0 0 0 0 0]
```

02 →

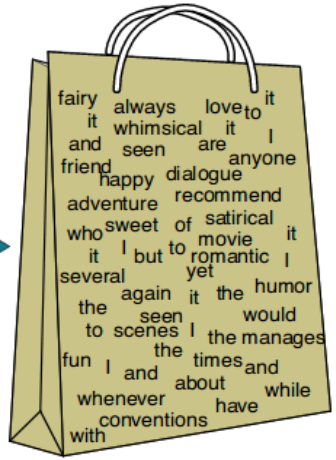
Bag of Words (BoW)

What is BoW?

Bag of Words represents text based on:

- **Word occurrence**
- **Word frequency**
  - It ignores grammar and word order and focuses only on **what words appear and how often**.

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!



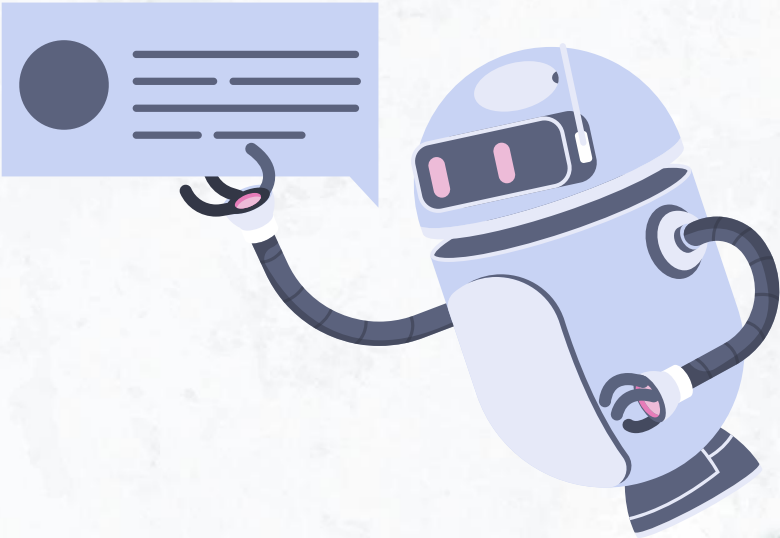
it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
…	…

	the	red	dog	cat	eats	food
1. the red dog →	1	1	1	0	0	0
2. cat eats dog →	0	0	1	1	1	0
3. dog eats food →	0	0	1	0	1	1
4. red cat eats →	0	1	0	1	1	0

Code:

```python
from sklearn.feature_extraction.text import CountVectorizer

documents = [
    "I really love studying natural language processing and machine learning",
    "Natural language processing is an important field in artificial intelligence",
    "I love learning machine learning and natural language processing"
]

# Create Bag of Words model
vectorizer = CountVectorizer()

# Fit and transform documents
X = vectorizer.fit_transform(documents)

# Print vocabulary
print("Vocabulary:")
print(vectorizer.get_feature_names_out())

# Print Bag of Words matrix
print("\nBag of Words Matrix:")
print(X.toarray())
```

Output:

```
Vocabulary:
['an' 'and' 'artificial' 'field' 'important' 'in' 'intelligence' 'is'
 'language' 'learning' 'love' 'machine' 'natural' 'processing' 'really'
 'studying']

Bag of Words Matrix:
[[0 1 0 0 0 0 0 0 1 1 1 1 1 1 1 1]
 [1 0 1 1 1 1 1 1 1 0 0 0 1 1 0 0]
 [0 1 0 0 0 0 0 0 1 2 1 1 1 1 0 0]]
```

THANK YOU